

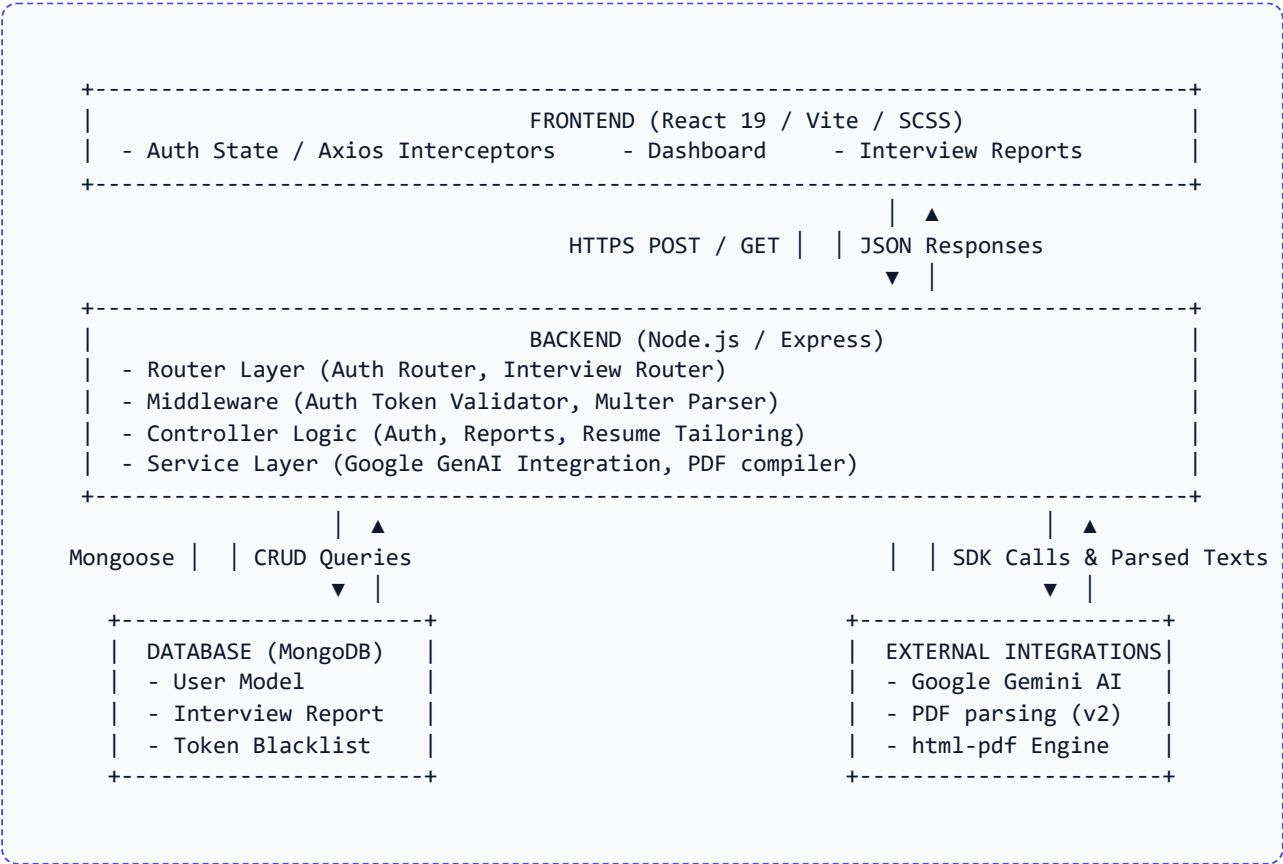
INTERVIEW AI - DEEP PROJECT GUIDE

A Complete Architecture walkthrough, Design Rationale, and 20 Critical Interview QA

1. Project Vision & High-Level Architecture

Interview AI is a full-stack, enterprise-grade AI-powered preparation suite designed to align candidate profiles with specific job descriptions, build interactive prep roadmaps, identify skill gaps, and dynamically tailor and compile resume PDFs.

The architecture adheres strictly to the classic **Client-Server-Database** design model, leveraging clean separation of concerns and custom RESTful endpoints.



2. Deep-Dive of Core Components

Frontend Architecture

- **React 19 & Vite:** Lightning-fast compilation, native React 19 concurrent features, and optimized single-page routing via React Router 7.
- **Axios Configuration:** Centralized instances configured with `withCredentials: true` to ensure seamless transfer of HTTP-only JWT cookies to and from the backend server.
- **SCSS Design Tokens:** Uses modular stylesheets containing variable structures, mixins, and premium glassmorphic UI layout definitions.

Backend & Services

- **Express Controller-Service Pattern:** Isolates routing definitions from business logic. Heavy lifting (AI prompt generation, PDF parsing, PDF rendering) is offloaded to reusable services.
- **Robust Auth Engine:** Validates user credentials using `bcryptjs`, generates JWT signatures, and sends them to the client via `httpOnly` cookies, protecting against Cross-Site Scripting (XSS) attacks. Supports secure logout through MongoDB token blacklisting.
- **In-Memory Parsing:** Multer stores uploads in memory buffers. The backend feeds the buffer directly to `pdf-parse`, extracts resume text, and frees up memory structures immediately using custom garbage-collection triggers.

3. Database Schema Blueprint

We use three core MongoDB collections managed via Mongoose schemas. Below are the key structures:

A. User Schema (`users`)

Stores core user data. Encrypts passwords using one-way bcrypt hashing before storage.

```
{
  username: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
}
```

B. Interview Report Schema (`InterviewReport`)

Houses job-specific analysis, custom interview roadmaps, questions, and match scores. Integrates relations using mongoose `ObjectId` references.

```
{
  jobDescription: { type: String, required: true },
  resume: { type: String },
  selfDescription: { type: String },
  matchScore: { type: Number, min: 0, max: 100 },
  technicalQuestions: [{ question: String, intention: String, answer: String }],
  behavioralQuestions: [{ question: String, intention: String, answer: String }],
  skillGaps: [{ skill: String, severity: { type: String, enum: ["low", "medium", "high"] } }],
  preparationPlan: [{ day: Number, focus: String, tasks: [String] }],
  user: { type: Schema.Types.ObjectId, ref: "users" },
  title: { type: String, required: true }
}
```

C. Blacklist Schema (`blacklistTokens`)

Maintains a list of invalidated tokens. An index is configured to run on MongoDB to automatically clear tokens after expiration (TTL - Time to Live).

```
{
  token: { type: String, required: true },
  createdAt: { type: Date, default: Date.now, expires: '1d' } // Auto-cleans expired tokens
}
```

4. Artificial Intelligence & Structured Prompts

Instead of receiving unformatted string outputs from LLMs, this project leverages the **Google GenAI SDK**'s JSON Schema generation using `zod-to-json-schema`.

By declaring a strict Zod validation schema on the server, we build the exact JSON output blueprint. Gemini is instructed to map its content explicitly to this structure using the `gemini-3-flash-preview` model. This eliminates parsing errors, protects client rendering from missing attributes, and guarantees that backend validation remains robust.

20 HIGH-IMPACT INTERVIEW QUESTIONS & ANSWERS

Curated specifically for demonstrating engineering depth on this project

BACKEND Q1: Explain the authentication flow in your project. Why did you choose HTTP-Only Cookies over LocalStorage for JWT storage?

Answer: Our auth flow uses JWTs. When users log in, the backend signs a token containing user details and sends it to the client via an HTTP-Only, Secure cookie. We chose HTTP-Only cookies because they are inaccessible to client-side JavaScript, which protects the token against Cross-Site Scripting (XSS) attacks. LocalStorage is vulnerable to XSS; any malicious third-party script running in the browser can read LocalStorage keys. With HTTP-Only cookies, the browser automatically attaches the cookie to subsequent API requests, maintaining a secure session.

BACKEND Q2: How does the logout mechanism work in your project? Explain the role of Token Blacklisting.

Answer: Since JWTs are stateless, clearing the client's cookie doesn't actually invalidate the token on the server. If a token is stolen, it could still be used until it expires. To address this, we implemented a Token Blacklisting mechanism. When a user logs out, we extract the current JWT, save it to a `blacklistTokens` collection in MongoDB, and clear the client cookie. Any incoming request is verified against the blacklist. To prevent the database from growing indefinitely, we configured a TTL (Time-To-Live) index on the `createdAt` field in the schema so that expired blacklisted tokens are deleted automatically.

AI & PROMPTS Q3: How do you guarantee that the Google Gemini API returns structured JSON data instead of raw text, and how do you validate it?

Answer: We use the Google GenAI SDK (`@google/genai`) combined with Zod. We define a strict validation schema using `zod` . We convert this schema into a standard JSON schema using `zod-to-json-schema` and supply it inside the `config.responseSchema` field during our Gemini API call, along with setting `responseMimeType: "application/json"` . This forces the Gemini model to return a structured JSON response matching our schema, ensuring attributes like `matchScore`, `skillGaps`, and `questions` are always structured properly.

BACKEND**Q4: How did you implement file uploading for resumes, and how did you parse them?**

Answer: We configured **Multer** to use memory storage (`multer.memoryStorage()`) instead of writing files to the server's disk. This allows the backend to hold the PDF file inside a buffer temporarily. We then pass this buffer to `pdf-parse` , which extracts the clean text content from the resume. Immediately after parsing, we release the memory structures by calling the parser's `destroy()` method to ensure memory safety.

BACKEND Q5: Explain the memory leak issue in pdf-parse v2 and how you addressed it.

Answer: Standard PDF parsing engines create persistent web worker processes and internal memory tables in the Node runtime to handle heavy PDFJS operations. In `pdf-parse` v2, these structures are not automatically garbage collected, leading to memory leaks over multiple runs. We addressed this by ensuring that we always call the `parser.destroy()` method immediately after extracting text. This releases the internal JS memory and destroys any dangling worker structures, keeping Node memory usage low.

DATABASE Q6: Why did you choose MongoDB over a relational database like PostgreSQL for this application?

Answer: The primary data structures generated by our application—namely the interview preparation reports, custom technical/behavioral questions, and day-to-day study plans—are highly hierarchical, flexible, and variable in size. In a relational database, storing nested arrays of questions and task lists would require complex joins across multiple tables. MongoDB allows us to store an entire Interview Report as a single document with nested sub-documents, enabling extremely fast reads and writes.

BACKEND Q7: What is the CORS configuration in your Express application, and why is it critical?

Answer: Since our frontend (Vite/React) and backend (Express) run on different ports during development, we must configure Cross-Origin Resource Sharing (CORS). We configure the `cors` middleware in Express with:

```
cors({
  origin: "http://localhost:5173",
  credentials: true
})
```

Setting `credentials: true` is critical; without it, the browser will refuse to send or receive the HTTP-Only authentication cookie, breaking our login session management.

FRONTEND**Q8: How does Axios handle credentials in the frontend? Show how you set up your Axios instance.**

Answer: By default, Axios does not send credentials (cookies) for cross-origin requests. To ensure every API call carries the JWT cookie, we create a custom axios instance:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true
});
```

This encapsulates the configuration so we don't have to repeat `withCredentials` in every individual request.

AI & PROMPTS**Q9: What strategies did you use to prevent LLM hallucination and ensure quality interview answers?**

Answer: We used system instructions and contextual prompt boundary constraints. We structured our prompt to feed the exact job description as the 'ground truth' requirements and the candidate's resume/self-description as the profile source. In the system instruction, we explicitly stated: "Analyze only the skills mentioned in the resume against the job description. Do not assume or invent achievements not listed." By restricting output fields using Zod's `.describe()` constraints, the model stays focused on authentic skill gaps and real questions.

AI & PROMPTS**Q10: What model of Google Gemini did you use and why? How did you handle API quota limits (429 Rate Limits)?**

Answer: We utilized the `gemini-3-flash-preview` model. It is optimized for low-latency JSON generation, cost-efficient, and supports structured JSON outputs perfectly. To handle rate limiting (429 errors), we: 1. Implemented client-side loading indicators and request-debouncing on the UI to prevent double-submissions. 2. Set up robust try-catch handlers on the backend to capture rate-limit codes and return informative error messages. 3. Configured an exponential backoff retry wrapper on the backend service for key AI calls.

BACKEND**Q11: How is the dynamic HTML-to-PDF resume generation implemented?**

Answer: When a user requests a tailored resume, our backend service sends the job description and parsed resume content to Gemini, asking it to generate a clean, tailored single-page resume formatted in standard HTML. We then feed this compiled HTML string into our PDF generation service (which uses `html-pdf` or `puppeteer`) to compile the HTML into a raw PDF Buffer. The Express controller then sets the response headers (`Content-Type: application/pdf`) and sends the buffer directly back to the client, allowing the user to download it immediately.

Q12: Why did you choose in-memory storage for Multer instead of disk storage? What are the tradeoffs?

Answer: We chose in-memory storage (`multer.memoryStorage()`) because resumes are lightweight (usually under 2MB), and writing them to the server disk requires cleanup scripts, risks exposing user files on the server file system, and incurs slow disk I/O. **Tradeoffs:** Disk storage is safer for huge uploads (e.g., videos) because memory storage consumes Node.js RAM. For our resume parsing scope, memory storage is faster, cleaner, and highly secure.

DATABASE **Q13: How did you design database indexes in your models to optimize search query speeds?**

Answer: We designed three critical indexes: 1. A `unique` index on the `email` field of the User model to make login lookups `O(1)`. 2. A compound index on the Interview Report model (`user: 1, createdAt: -1`) to optimize retrieving user dashboards sorted by latest reports. 3. A TTL (Time-To-Live) index on the Blacklist model's `createdAt` field, instructing MongoDB to automatically delete token documents after 24 hours.

FRONTEND **Q14: How does React 19 improve state management or form handling in your project?**

Answer: React 19 introduces native hooks like `useActionState` and `useFormStatus`, which simplify asynchronous transitions and pending state management for form submits (e.g., logging in or uploading a resume). Instead of manually tracking `isLoading` and `error` states via hooks, we wrap asynchronous actions in React transitions, leading to cleaner components and fewer re-renders.

BACKEND **Q15: How do you secure Express routes so that only authenticated users can access them?**

Answer: We built an authorization middleware called `authMiddleware`. It checks for the presence of the JWT token in cookies. If the token is found, it decodes and verifies the JWT. If validation succeeds, it looks up the user, attaches the user object to `req.user`, and calls `next()` to hand off control to the controller. If the token is invalid, expired, or blacklisted, it returns a `401 Unauthorized` response, preventing unauthorized access.

BACKEND **Q16: How do you handle password security and storage in your database?**

Answer: We never store passwords in plain text. We use `bcryptjs` to hash passwords. During registration, we generate a salt and hash the password using `bcrypt.hash()`. During login, we fetch the hashed password from the database and use `bcrypt.compare()` to verify if the incoming password matches the hash. Bcrypt uses a slow-hashing algorithm, protecting passwords from brute-force and rainbow table attacks.

ARCHITECTURE Q17: Describe your error-handling middleware strategy on the Express backend.

Answer: We centralize all runtime errors using a global error-handling middleware. Instead of scattered try-catch logs, our controllers pass caught errors to `next(error)`. Our global handler intercepts them, logs the stack trace in development, and returns clean, uniform JSON payloads to the frontend:

```
app.use((err, req, res, next) => {
  res.status(err.statusCode || 500).json({
    success: false,
    message: err.message || "Internal Server Error"
  });
});
```

This keeps client responses consistent and ensures server-side failures don't leak implementation details.

FRONTEND Q18: What styling strategy did you use for the UI, and how did you organize your stylesheet structures?

Answer: We used Sass/SCSS with a structured design token system. We declared visual variables (colors, typography scales, glassmorphism shadows) in a centralized `_variables.scss` file. We then structured styles by features using SCSS mixins and responsive breakpoints. This modular approach allows clean scaling without repeating styles, keeping the UI visually cohesive.

BEHAVIORAL Q19: How did you prioritize features when building this project? What was your MVP scope?

Answer: I defined the Minimum Viable Product (MVP) based on the core user problem: getting immediate feedback on resumes against target jobs. - **MVP Phase:** Resume parsing, Gemini matching, and generating the preparation plan. - **Phase 2:** Authentication and saving reports (MERN integration). - **Phase 3:** Tailored single-page resume generation and PDF download. This iterative prioritization allowed me to validate the core AI logic first before building out session management and styling panels.

BEHAVIORAL Q20: What was the biggest technical challenge you faced in this project, and how did you resolve it?

Answer: The biggest challenge was getting consistent single-page PDF resumes from AI-generated HTML. The Gemini API would sometimes generate long structures that wrapped onto two pages, or the styling would break during HTML-to-PDF compilation because of PhantomJS/html-pdf rendering limitations. I resolved this by: 1. Refining the prompt schema to mandate a strict single-page grid structure. 2. Adjusting page margins and styling to use inline flexboxes and CSS grids instead of floating layouts. 3. Switching our backend rendering to use Headless Puppeteer for printing, as Puppeteer executes standard Chromium layouts which cleanly compile CSS styles to PDF.